

# 软件安全第七次实验报告

姓名：林晖鹏 学号：2312966

## 一、实验名称：

AFL模糊测试实验

## 二、实验要求：

根据课本7.4.5章节，复现AFL在KALI下的安装、应用，查阅资料理解覆盖引导和文件变异的概念和含义。

## 三、实验过程：

### 1. 安装AFL

按照书上的指令安装AFL。

代码块

```
1 sudo apt-get install afl
```

发现afl已经被afl++替代，找不到afl这个包了，所以我们需要手动安装经典版本afl。

先安装依赖包：

代码块

```
1 sudo apt install build-essential clang
```

然后下载并解压AFL

代码块

```
1 wget https://github.com/google/AFL/archive/refs/tags/2.52b.tar.gz
2 tar -xvzf 2.52b.tar.gz
3 cd AFL-2.52b
```

但是我们发现，kali里面连不上外网，为了方便，我们就不再里面搭梯子了，而是选择在外面系统下载之后复制进kali虚拟机里面。

```
HTTP request sent, awaiting response... 404 Not Found
2025-04-16 22:10:14 ERROR 404: Not Found.
```

我们前往github下载完之后，传入kali虚拟机，然后进行解压，并安装到系统目录下，这里下载的是32位的软件，可是我的kali是64位的，需要跳过一些设置，这里不作赘述。

我们输入 `ls /usr/bin/afl *` 检查路径是否存在afl。

```
(kali㉿kali)-[~]
└─$ ls /usr/bin/afl*
/usr/bin/afl-analyze      /usr/bin/afl-cmin      /usr/bin/afl-gotcpu.c
/usr/bin/afl-analyze.c    /usr/bin/afl-fuzz      /usr/bin/afl-plot
/usr/bin/afl-as           /usr/bin/afl-fuzz.c    /usr/bin/afl-showmap
/usr/bin/afl-as.c         /usr/bin/afl-g++       /usr/bin/afl-showmap.c
/usr/bin/afl-as.h         /usr/bin/afl-gcc       /usr/bin/afl-tmin
/usr/bin/afl-clang        /usr/bin/afl-gcc.c     /usr/bin/afl-tmin.c
/usr/bin/afl-clang++      /usr/bin/afl-gotcpu    /usr/bin/afl-whatsup
```

验证完毕，已经成功安装到目录下。

## 2. 应用AFL

### 2.1 创建测试文件

我们创建一个test.c文件，并写入代码。

这个代码是如果程序传入“deadbeef”，程序就会终止，否则就输出传入的字符。

代码块

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char **argv) {
4     char ptr[20];
5     if(argc>1){
6         FILE *fp = fopen(argv[1], "r");
7         fgets(ptr, sizeof(ptr), fp);
8     }
9     else{
```

```

10         fgets(ptr, sizeof(ptr), stdin);
11     }
12     printf("%s", ptr);
13     if(ptr[0] == 'd') {
14         if(ptr[1] == 'e') {
15             if(ptr[2] == 'a') {
16                 if(ptr[3] == 'd') {
17                     if(ptr[4] == 'b') {
18                         if(ptr[5] == 'e') {
19                             if(ptr[6] == 'e') {
20                                 if(ptr[7] == 'f') {
21                                     abort();
22                                 }
23                                 else
24                                     }
25                                 else    printf("%c",ptr[6]);
26                             }
27                             else    printf("%c",ptr[5]);
28                         }
29                         else    printf("%c",ptr[4]);
30                     }
31                     else    printf("%c",ptr[3]);
32                 }
33                 else    printf("%c",ptr[2]);
34             }
35             else    printf("%c",ptr[1]);
36         }
37         else    printf("%c",ptr[0]);
38         return 0;
39     }

```

## 2.2 编译测试文件

我们输入指令 `afl-gcc -o test 文件路径` 来编译这个c文件，得到可执行文件test。

```

(kali@kali)-[~]
$ afl-gcc -o test '/home/kali/Documents/demo1/test.c'
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 14 locations (64-bit, non-hardened mode, ratio 100%).

```

接着我们输入 `readelf -s ./test | grep afl` 来查看插桩符号：

```
(kali@kali)-[~]
$ readelf -s ./test | grep afl
 4: 0000000000001630      0 NOTYPE  LOCAL  DEFAULT 15 __afl_maybe_log
 6: 0000000000004090      8 OBJECT  LOCAL  DEFAULT 26 __afl_area_ptr
 7: 0000000000001660      0 NOTYPE  LOCAL  DEFAULT 15 __afl_setup
 8: 0000000000001640      0 NOTYPE  LOCAL  DEFAULT 15 __afl_store
 9: 0000000000004098      8 OBJECT  LOCAL  DEFAULT 26 __afl_prev_loc
10: 0000000000001658      0 NOTYPE  LOCAL  DEFAULT 15 __afl_return
11: 00000000000040a8      1 OBJECT  LOCAL  DEFAULT 26 __afl_setup_failur
12: 0000000000001681      0 NOTYPE  LOCAL  DEFAULT 15 __afl_setup_first
14: 0000000000001946      0 NOTYPE  LOCAL  DEFAULT 15 __afl_setup_abort
15: 000000000000179b      0 NOTYPE  LOCAL  DEFAULT 15 __afl_forkserver
16: 00000000000040a4      4 OBJECT  LOCAL  DEFAULT 26 __afl_temp
17: 0000000000001859      0 NOTYPE  LOCAL  DEFAULT 15 __afl_fork_resume
18: 00000000000017c1      0 NOTYPE  LOCAL  DEFAULT 15 __afl_fork_wait_lo
19: 000000000000193e      0 NOTYPE  LOCAL  DEFAULT 15 __afl_die
20: 00000000000040a0      4 OBJECT  LOCAL  DEFAULT 26 __afl_fork_pid
62: 00000000000040b0      8 OBJECT  GLOBAL  DEFAULT 26 __afl_global_area_ptr
```

然后我们还需要输入下面指令，让系统将coredumps输为文件。

代码块

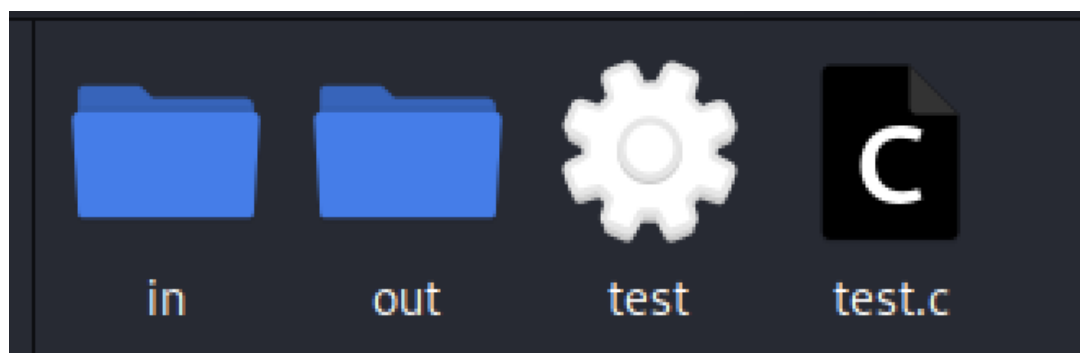
```
1 sudo sh -c 'echo core > /proc/sys/kernel/core_pattern'
```

## 2.3 创建测试用例

先创建文件夹in和out，然后在输入文件夹里面创建一个包含字符串hello的文件，分别输入下面这些指令即可：

代码块

```
1 mkdir in out
2 echo hello> in/foo
```



## 2.4 启动模糊测试

我们输入 `afl-fuzz -i in -o out -- ./test @@` 来启动模糊测试。

```
american fuzzy lop 2.57b (test)

process timing
  run time : 0 days, 0 hrs, 4 min, 18 sec
  last new path : 0 days, 0 hrs, 3 min, 44 sec
  last uniq crash : 0 days, 0 hrs, 3 min, 34 sec
  last uniq hang : none seen yet
  cycle progress
    now processing : 4 (50.00%)
    paths timed out : 0 (0.00%)
  stage progress
    now trying : havoc
    stage execs : 60/512 (11.72%)
    total execs : 907k
    exec speed : 3055/sec
  fuzzing strategy yields
    bit flips : 2/320, 2/312, 0/296
    byte flips : 0/40, 0/32, 0/16
    arithmetics : 1/2240, 0/70, 0/0
    known ints : 0/222, 1/863, 0/703
    dictionary : 0/0, 0/0, 0/0
      havoc : 2/503k, 0/399k
      trim : 48.94%/10, 0.00%

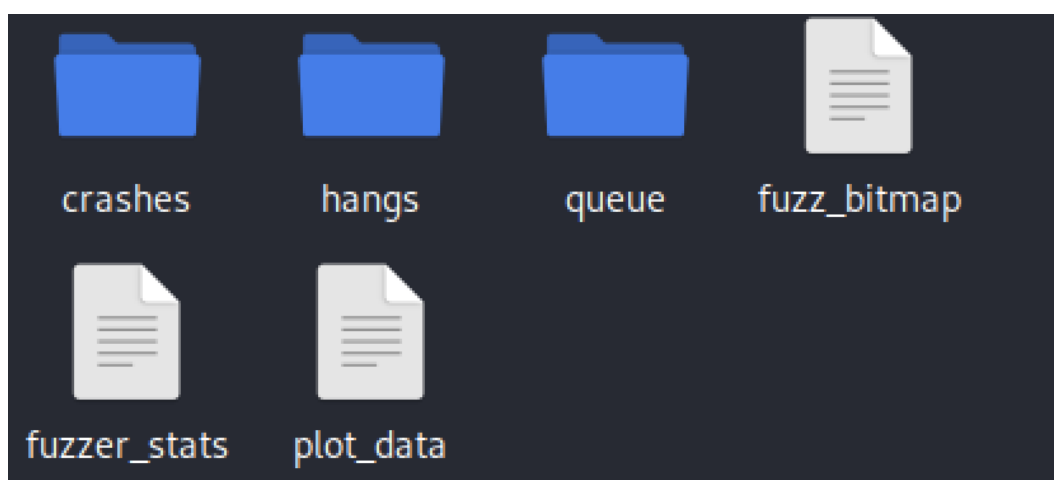
overall results
  cycles done : 131
  total paths : 8
  uniq crashes : 1
  uniq hangs : 0

map coverage
  map density : 0.01% / 0.03%
  count coverage : 1.00 bits/tuple
  findings in depth
    favored paths : 8 (100.00%)
    new edges on : 8 (100.00%)
    total crashes : 1 (1 unique)
    total tmouts : 1 (1 unique)
  path geometry
    levels : 6
    pending : 0
    pend fav : 0
    own finds : 7
    imported : n/a
    stability : 100.00%

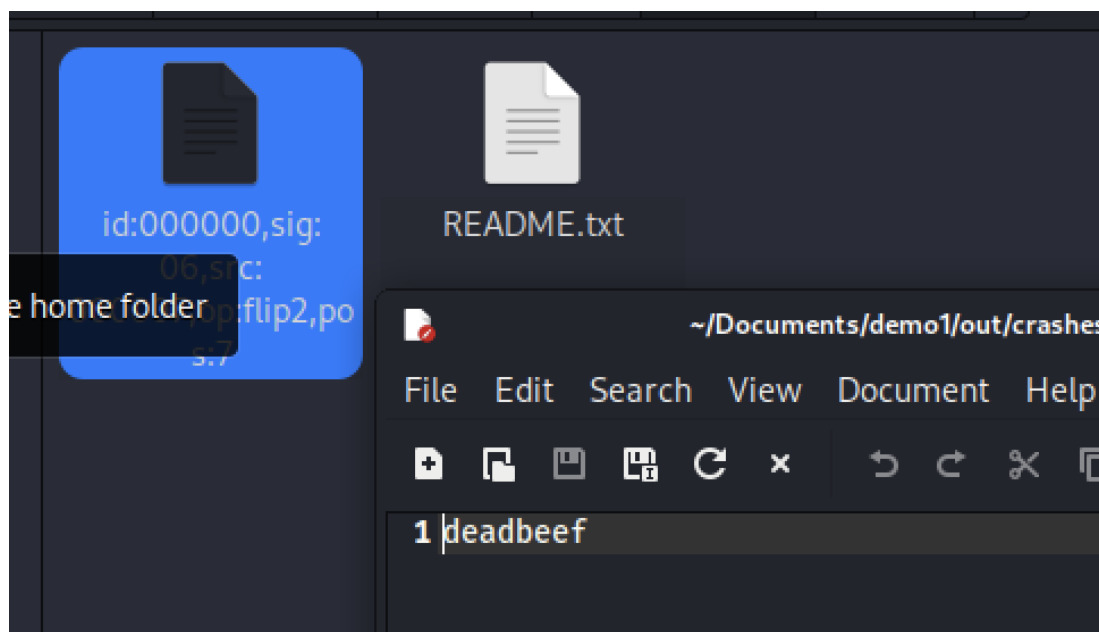
[cpu000: 89%]
```

## 2.5 crash分析

查看out文件夹，发现生成了很多文件。



我们查看crashes文件夹，可以看到有问题的输入案例。正好符合程序的设计。



我们尝试输入这个有问题的字符串给程序，发现确实出现错误，验证完毕！

```
(kali㉿kali)-[~/Documents/demo1]  
$ ./test "deadbeef"  
zsh: segmentation fault ./test "deadbeef"
```

---

## 四、覆盖引导和文件变异

### 1. 覆盖引导

**覆盖引导**是指 AFL 在生成测试输入时，会根据程序执行路径（即“覆盖率”）的变化来引导后续输入的生成。

- a. 在编译文件的时候，会插桩记录路径分支，以便于计算路径覆盖率。
- b. 每个测试用例执行之后，会记录其路径。
- c. 如果某个输入的路径为之前没走过的，就会标记这个路径，后续的文件变异会在这个输入的基础上进行变异

这种方式促使 AFL 不断探索新的代码路径，尤其是深层逻辑，从而提高找到崩溃或漏洞的概率。

### 2. 文件变异

**文件变异**指的是对原始输入文件（通常是合法的种子输入）进行各种形式的**字节级修改**，以产生新的测试用例。

### 3. 理解

总的来说，覆盖引导和文件变异好比与，前者是发现有能开其他门的钥匙之后，在这个钥匙的模版上进行修改，希望得到更好的钥匙能开更多不同的门；后者是不断修改钥匙，尝试不同的方案。

---

## 五、实验总结

- 在本实验中，我们尝试用AFL对一份代码进行模糊测试，并成功测试出漏洞用例。
  - 同时熟悉了AFL的使用，在使用的过程中，也对模糊测试的理解更加深入，更加深入的理解了覆盖引导和文件变异的思想，对渗透测试有了新的感悟。
  - 并且首次接触kali虚拟机，在配置环境过程中，熟悉了kali上面linux终端的使用。
-

